Non-uniform QPSE

Convention

- Arbitrary quadratic form of a pair of data vector, x_A and x_B, defined with matrix $\mathbf{E}$:

  $$q = x_B^\dagger \mathbf{E} x_A = \mathrm{Tr}\left(\mathbf{E}\, x_A x_B^\dagger\right)$$

- Data Covariance:

  $$\langle x_A x_B^\dagger \rangle = \mathbf{N}^{AB} + \sum_\alpha p_\alpha \mathbf{Q}_\alpha^{AB}$$

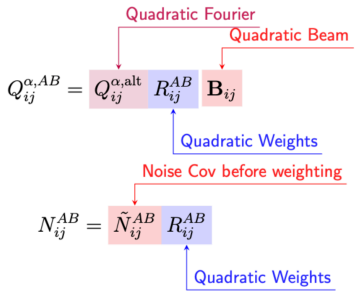
- Expectation value of q:

  $$\begin{aligned}\langle q \rangle &= \mathrm{Tr}\left(\mathbf{E}\,\langle x_A x_B^\dagger \rangle\right) \\ &= \sum_\alpha p_\alpha \mathrm{Tr}\left(\mathbf{E}\,\mathbf{Q}_\alpha^{AB}\right) + \mathrm{Tr}\left(\mathbf{E}\,\mathbf{N}^{AB}\right)\end{aligned}$$

  which is a linear combination of the bandpower p_α's.

HERA_PSPEC

- **Response matrix.** Let x_A (and x_B) denote the (flag + tapper + arbitrary weights) weighted data and p_α denote the delay space band-power of signal. Then we can decompose the response matrix, $\mathbf{Q}_\alpha^{AB}$, as the following elementment-wise expression:




```python
def generate_response_matrix(quad_Fourier_list, quad_Weights, quad_Beam):
    """
    Vectorized version - eliminates the loop entirely
    """
    # Convert list to 3D array: (n_modes, n_freqs, n_freqs)
    quad_Fourier_array = np.array(quad_Fourier_list)

    # Broadcast multiplication: aux is (n_freqs, n_freqs), quad_Fourier_array is (n_modes
    aux = quad_Weights * quad_Beam
    resp_mat_array = aux[None, :, :] * quad_Fourier_array

    return resp_mat_array
```

where i, j denote the frequencies ν_i, ν_j.

- **Quadratic Fourier**:

  Consistent with the existing $\mathbf{Q}^{\alpha,\mathrm{alt}}$ definition in hera_pspec.

```
def get_quad_Fourier_list(Ndlys, d_eta):

    Q_alt_li = []
    Nfreqs = Ndlys

    for mode in range(Ndlys):

        if Ndlys % 2 == 0:
            start_idx = -Ndlys/2
        else:
            start_idx = -(Ndlys - 1)/2
        m = (start_idx + mode) * np.arange(Nfreqs)
        m = np.exp(-2j * np.pi * m / Ndlys) * d_eta
        Q_alt = np.einsum('i,j', m.conj(), m) # dot it with its conjugate
        Q_alt_li.append(Q_alt)

    return Q_alt_li
```

The quadratic term of Fourier transform in the α-th delay bin:

$$Q_{ij}^{\alpha,\mathrm{alt}} \equiv \sum_{a\in\{\alpha\}} \lambda_a(\nu_i)\lambda_a^*(\nu_j)$$

If no LST binning, it represents a quadratic Fourier term:

$$Q_{ij}^{a,\mathrm{alt}} \equiv \lambda_a(\nu_i)\lambda_a^*(\nu_j)$$

- **Quadratic Beam**:

  Defined as the integral of the multiplication of two normalised beams.

$$\begin{aligned}\mathbf{B}_{ij} &\equiv \frac{1}{\Omega_p(\nu_i)\Omega_p(\nu_j)} \int \mathrm{d}^2\boldsymbol{u}\, \tilde{A}(\boldsymbol{u_b}-\boldsymbol{u},\nu_i)\tilde{A}(\boldsymbol{u_b}-\boldsymbol{u},\nu_j) \\ &= \frac{1}{\Omega_p(\nu_i)\Omega_p(\nu_j)} \int \mathrm{d}^2\boldsymbol{\theta}\, A(\boldsymbol{\theta},\nu_i)A(\boldsymbol{\theta},\nu_j)\end{aligned}$$

where Ω_p is the beam normalisation.

```
def generate_quad_Beam(normalised_beam, d_Omega=1.0, isotropic_signal=False):
    """
    normalised_beam: shape (N_freqs, N_directions) - Beam response normalized by Omega_p
        dType: np.ndarray (real or complex with zero imaginary part)
    d_Omega: float, solid angle element for each direction
    isotropic_signal: bool, if True, assumes isotropic signal and integrates over directions first
        This is just for testing purposes

    Returns:
    quad_Beam: shape (N_freqs, N_freqs)
    """
    if not isotropic_signal:
        # Vectorized computation: sum over directions
        quad_Beam = np.einsum('if,jf->ij', normalised_beam, normalised_beam) * d_Omega
        return quad_Beam
    else:
        normalised_beam_int = np.sum(normalised_beam, axis=1) * d_Omega
        quad_Beam = np.einsum('i,j->ij', normalised_beam_int, normalised_beam_int)
        return quad_Beam
```

- **Quadratic weights**:

  The quadratic form of arbitrary weights R_{ij}^{AB} describes the frequency axis weighting for x_A and x_B: $R_{ij}^{AB} = \tilde{R}_{A,i}\tilde{R}_{B,j}^*$,

  where $\tilde{R}_{A,i}$ can be seen as an arbitrary tapering function that also serves to flag data.

Generate $\tilde{R}_{A,i}$:

```
def taper_apodised_flags(flag_arr, ramp_width, taper_coeffs=None):
    """
    Combines apodization around flags with a global tapering window.
    flag_arr : boolean array (True = flagged, False = good)
    ramp_width : integer number of samples for the half-cosine ramp on each side
    taper_coeffs : optional float array in [0,1] same length as flag_arr
                   if None, uses Blackman-Harris window
    returns apod_taper : float array in [0,1] same length as flag_arr
    """
    n = len(flag_arr)
    if taper_coeffs is None:
        import uvtools.dspec as dspec
        taper_coeffs = dspec.gen_window('blackman-harris', n)

    apod = apodize_around_flags(flag_arr, ramp_width)

    apod_taper = apod * taper_coeffs

    # small numerical safety
    apod_taper = np.clip(apod_taper, 0.0, 1.0)
    return apod_taper
```

This function takes taper weights and a flag array, and generates $\tilde{R}_{A,i}$, the **apodized** taper weights.

PSE

- Estimator:

  $$\langle q_\alpha \rangle = \frac{1}{M^{AB}} \mathrm{Tr}\left(\mathbf{E}\,\langle x_A x_B^\dagger \rangle\right) - b^{AB} = \sum_\alpha p_\alpha W_\alpha^{AB}$$

  where $M^{AB} = \sum_\alpha \mathrm{Tr}\left(\mathbf{E}\,\mathbf{Q}_\alpha^{AB}\right)$, $W^{AB} = \mathrm{Tr}\left(\mathbf{E}\,\mathbf{Q}_\alpha^{AB}\right)/M^{AB}$,

  and $b^{AB} = \mathrm{Tr}\left(\mathbf{E}\,\mathbf{N}^{AB}\right)/M^{AB}$.

- To estimate p_α, a common practice is to define the quadratic form with the response matrix $\mathbf{E} = \mathbf{Q}_\alpha^{AB}$.

- However, HERA_pspec actually uses $\mathbf{Q}^{\alpha,\mathrm{alt}}$.
  This modification has little impact on mode mixing when no flagging is applied; however, it performs significantly worse in the presence of arbitrary weighting with flags.

```python
def p_hat(self, vis1, vis2, flag1, flag2, N_cov=None, weight1=None, weight2=None, ramp_width=20, quad_form="Q"):
    """
    weight1, weight2: shape (N_freqs,)
    quad_form: "Q" or "Q_alt" or provided matrices
    """
    N_cov = N_cov

    if weight1 is None:
        weight1 = taper_apodised_flags(flag1, ramp_width, taper_coeffs=self.taper_func)
    if weight2 is None:
        weight2 = taper_apodised_flags(flag2, ramp_width, taper_coeffs=self.taper_func)

    weighted_vis1 = vis1 * weight1
    weighted_vis2 = vis2 * weight2
    Dmat = np.einsum('i,j->ij', weighted_vis1, weighted_vis2.conj())

    quad_Weights = np.einsum('i,j->ij', weight1, weight2.conj())
    self.resp_mat_arr = generate_response_matrix(self.quad_Fourier_list, quad_Weights, self.quad_Beam)


    if quad_form == "Q":
        quad_form_matrices = self.resp_mat_arr
    elif quad_form == "Q_alt":
        quad_form_matrices = self.quad_Fourier_list
    else:
        quad_form_matrices = quad_form # Assume user provided valid matrices
    wind_coeffs_li = []
    norm_li = []
    bias_li = []
    p_hat_li = []
    for i in range(self.Ndlys):
        wind_coeffs, norm_factor = generate_window_func(quad_form_matrices[i], self.resp_mat_arr)
        wind_coeffs_li.append(wind_coeffs)
        bias = evaluate_bias(quad_form_matrices[i], quad_Weights, norm_factor, noise_cov=N_cov)
        p_hat_i = trace_AB(quad_form_matrices[i], Dmat) / norm_factor - bias
        p_hat_li.append(p_hat_i)
        bias_li.append(bias)
        norm_li.append(norm_factor)
```

- Links:
  All the code can be found at:
  Code only: https://github.com/zzhang0123/weighted_QPSE/blob/main/nonuniform_pspec.py
  Or code + demonstration: https://github.com/zzhang0123/weighted_QPSE/blob/main/test_nonuniform_weighting.ipynb

Classes

`QE_window`

Window function analysis for quadratic estimators.

Parameters:

- `freqs`: Frequency array [Hz]
- `normalised_beam`: Beam response normalized by Ω_p
- `d_Omega`: Solid angle element [steradians]

Methods:

- `generate_window_coeffs(weight1, weight2, quad_form="Q")`: Generate window functions

`nonuniform_pspec`

Full power spectrum estimation with non-uniform weighting.

Parameters:

- `freqs`: Frequency array [Hz]
- `normalised_beam`: Normalized beam pattern
- `d_Omega`: Solid angle element
- `taper`: Global tapering window type ('blackman-harris', 'none')
- `test_with_isotropic_signal`: Use isotropic signal assumption for testing

Methods:

- `p_hat(vis1, vis2, flag1, flag2, ...)`: Estimate power spectrum from visibilities